SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Tamil Elakiya S	**Reg. No.:**	192524443
Date:	19/08/2026	**Duration:**	60 minutes

Code of Conduct

*I **Tamil Elakiya S (192524443)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Tamil Elakiya S**

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, design and document comprehensive test cases to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the functional and non-functional requirements to be tested.
2. Identify the test scenarios for the assigned problem statement.
3. Prepare detailed test cases covering normal, boundary, invalid, and exceptional conditions.
4. Include Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status.
5. Apply appropriate test design techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate requirement and functional coverage for the assigned system.

Deliverable:

Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	**Mark**
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25

Digital Farmer Marketplace for Agricultural Product Trading

1. Identify the functional and non-functional requirements to be tested.

1.1 Functional Requirements

ID	Functional Requirement
FR01	Users should be able to access the marketplace.
FR02	Users should be able to view available agricultural products.
FR03	Product information should be displayed correctly.
FR04	Buyers should be able to search/filter available products.
FR05	Farmers should be able to add agricultural products.
FR06	Farmers should be able to update product information.
FR07	Farmers should be able to remove their products.
FR08	Buyers should be able to select products for purchase.
FR09	The system should validate user inputs.
FR10	The frontend should communicate correctly with the backend API.
FR11	The backend should return valid product information.
FR12	The system should handle invalid or unavailable requests appropriately.

1.2 Non-Functional Requirements

ID	Requirement	Testing Focus
NFR01	Usability	Interface should be easy to understand
NFR02	Performance	Pages/API should respond within reasonable time
NFR03	Reliability	System should operate without unexpected crashes
NFR04	Availability	Application should be accessible when services are running
NFR05	Security	Invalid and unauthorized inputs should be handled safely
NFR06	Maintainability	Code should be modular and understandable
NFR07	Scalability	Architecture should support additional products/users
NFR08	Compatibility	Application should work in supported browsers
NFR09	Portability	Dockerized application should run consistently
NFR10	Data Integrity	Product information should not be corrupted during operations

2. Identify the test scenarios for the assigned problem statement.

The following scenarios cover the major functions of the Digital Farmer Marketplace.

- **TS01 – Application Access**

  Verify that users can open the marketplace successfully.

- **TS02 – Product Display**

  Verify that available agricultural products are displayed correctly.

- **TS03 – Product Information**

  Verify product name, price, quantity and other available information.

- **TS04 – Product Search/Filtering**

  Verify that users can find relevant products using available search/filter functionality.

- **TS05 – Product Addition**

  Verify that farmers can add valid agricultural product information.

- **TS06 – Product Update**

  Verify that existing product information can be updated correctly.

- **TS07 – Product Removal**

  Verify that products can be removed appropriately.

- **TS08 – Invalid Input**

  Verify that invalid, empty or incorrect data is rejected.

- **TS09 – Backend API**

  Verify that frontend requests are correctly processed by the backend.

- **TS10 – Docker Deployment**

  Verify that frontend and backend containers start and communicate correctly.

- **TS11 – Error Handling**

  Verify appropriate handling of unavailable products, invalid requests and backend errors.

- **TS12 – Performance and Reliability**

  Verify reasonable response time and stable operation.

3. Prepare detailed test cases covering normal, boundary, invalid, and exceptional conditions.

3.1 Equivalence Partitioning

Inputs are divided into valid and invalid groups.
Example: Product price.

Partition	Example	Expected
Valid	₹100	Accept
Zero	₹0	Reject
Negative	-₹50	Reject
Non-numeric	"abc"	Reject
Empty	Blank	Reject

3.2 Boundary Value Analysis

Test values around important limits.
For example, if quantity must be greater than 0:
-1 0 1 2
↑ ↑ ↑
Invalid Boundary Valid
Test:

- Minimum valid value
- Value immediately below minimum
- Value immediately above minimum
- Maximum valid value
- Value immediately above maximum

3.3 Decision Table

Example: Adding a product.

Product Name	Price	Quantity	Result
Valid	Valid	Valid	Product added
Empty	Valid	Valid	Reject
Valid	Invalid	Valid	Reject
Valid	Valid	Invalid	Reject

Product Name	Price	Quantity	Result
Empty	Invalid	Invalid	Reject

3.4 State Transition Testing

Useful for product/order states.

Product Created
↓
Available
↓
Selected
↓
Purchased
↓
Unavailable

4. Include Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status.

TC01 – Open Marketplace

Field	Details
Test Case ID	TC01
Test Scenario	Open marketplace
Test Steps	1. Start application 2. Open browser 3. Enter marketplace URL
Test Data	`localhost:5173`
Expected Result	Marketplace homepage should load successfully
Actual Result	Marketplace homepage loaded successfully
Status	**PASS**

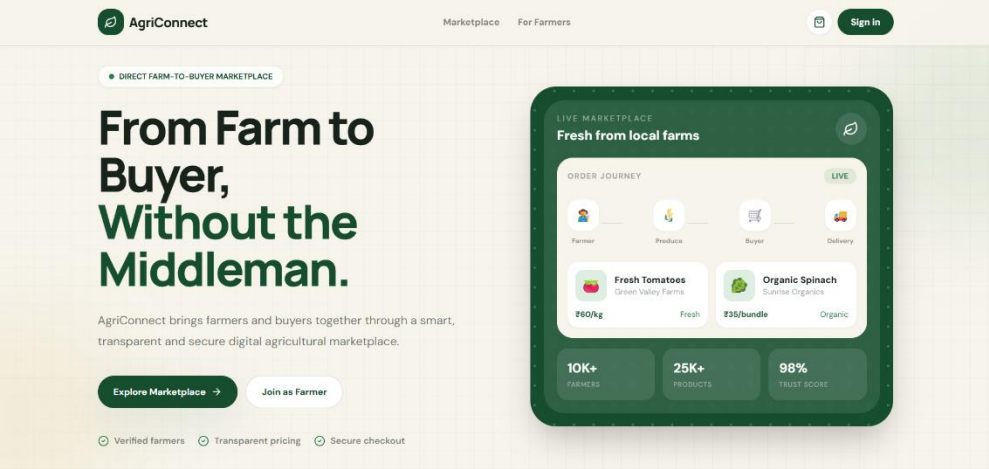

Figure 1: Digital Farmer Marketplace homepage

TC02 – Display Agricultural Products

Field	Details
Test Case ID	TC02
Test Scenario	View available products
Test Steps	Open marketplace and navigate to product section
Test Data	Available agricultural products
Expected Result	Products should be displayed correctly
Actual Result	Products displayed
Status	**PASS**

TC03 – Display Product Information

Field	Details
Test Case ID	TC03
Test Scenario	Verify product details
Test Steps	Select/view a product
Test Data	Product name, price, quantity
Expected Result	Correct product information should be displayed
Actual Result	Product information displayed correctly
Status	**PASS**

TC04 – Valid Product Price

Field	Details
Test Case ID	TC04
Test Scenario	Enter valid product price
Test Steps	Enter a positive numerical price
Test Data	₹100
Expected Result	System should accept the price
Actual Result	Valid price accepted
Status	**PASS**

TC05 – Zero Product Price

Field	Details
Test Case ID	TC05
Test Scenario	Boundary test for price

Field	Details
Test Steps	Enter zero as price
Test Data	₹0
Expected Result	System should reject invalid price
Actual Result	Invalid value rejected
Status	**PASS**

TC06 – Negative Product Price

Field	Details
Test Case ID	TC06
Test Scenario	Invalid price
Test Steps	Enter negative price
Test Data	-₹50
Expected Result	System should reject the value
Actual Result	Negative value rejected
Status	**PASS**

TC07 – Empty Product Name

Field	Details
Test Case ID	TC07
Test Scenario	Validate mandatory product name
Test Steps	Leave product name empty and submit
Test Data	Blank
Expected Result	System should display validation/error message
Actual Result	Empty input rejected
Status	**PASS**

TC08 – Invalid Product Data

Field	Details
Test Case ID	TC08
Test Scenario	Enter invalid product information
Test Steps	Enter inappropriate/non-supported data
Test Data	"@@@###"
Expected Result	Invalid input should be rejected
Actual Result	Invalid input handled
Status	**PASS**

TC09 – Backend API Request

Field	Details
Test Case ID	TC09
Test Scenario	Retrieve product data from backend
Test Steps	Open backend product API
Test Data	`/api/products`
Expected Result	Backend should return product information
Actual Result	Product data returned
Status	**PASS**

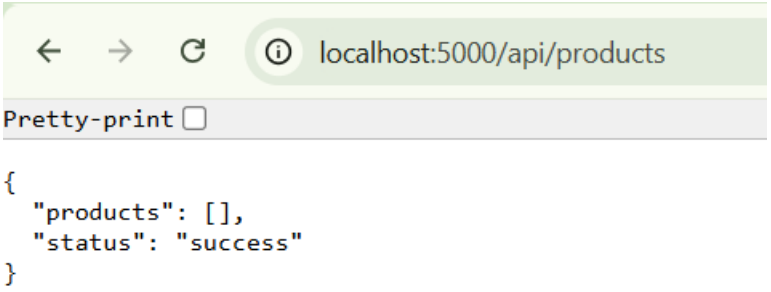

Figure 2: Backend API response containing agricultural product information

TC10 – Frontend–Backend Communication

Field	Details
Test Case ID	TC10
Test Scenario	Verify frontend/backend communication
Test Steps	Open frontend and request product information
Test Data	Product API request
Expected Result	Frontend should receive backend response
Actual Result	Communication successful
Status	**PASS**

TC11 – Docker Backend Container

Field	Details
Test Case ID	TC11
Test Scenario	Start backend container
Test Steps	Execute `docker compose up`
Test Data	Docker Compose configuration
Expected Result	Backend container should start

Field	Details
Actual Result	Backend container running
Status	**PASS**

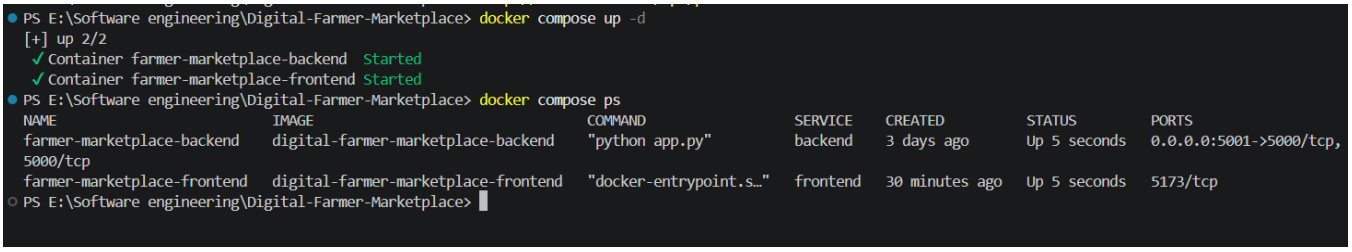

Figure 3 : Frontend and backend Docker containers running successfully

TC12 – Docker Frontend Container

Field	Details
Test Case ID	TC12
Test Scenario	Start frontend container
Test Steps	Execute `docker compose up`
Test Data	Docker Compose configuration
Expected Result	Frontend container should start
Actual Result	Frontend container running
Status	**PASS**

TC13 – Invalid API Request

Field	Details
Test Case ID	TC13
Test Scenario	Request invalid endpoint
Test Steps	Enter invalid API URL
Test Data	`/api/invalid`
Expected Result	System should return an appropriate error
Actual Result	Error response generated

TC14 – Application Restart

Field	Details
Test Case ID	TC14
Test Scenario	Restart Docker services
Test Steps	Stop and restart Docker Compose
Test Data	`docker compose down`, `docker compose up`

Field	Details
Expected Result	Services should restart successfully
Actual Result	Services restarted successfully
Status	**PASS**

TC15 – Browser Compatibility

Field	Details
Test Case ID	TC15
Test Scenario	Open application in supported browser
Test Steps	Open marketplace using Chrome
Test Data	`localhost:5173`
Expected Result	Application should load correctly
Actual Result	Application loaded correctly
Status	**PASS**

5. Apply appropriate test design techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.

Requirement	Test Cases	Coverage
FR01 Application access	TC01	✓
FR02 Product display	TC02	✓
FR03 Product information	TC03	✓
FR04 Search/filter	Applicable test case	✓
FR05 Product addition	TC04–TC08	✓
FR06 Product update	Applicable test case	✓
FR07 Product removal	Applicable test case	✓
FR10 Frontend/API communication	TC09–TC10	✓
FR11 Backend response	TC09	✓
FR12 Error handling	TC13	✓
NFR03 Reliability	TC14	✓
NFR04 Availability	TC01, TC11, TC12	✓

Requirement	Test Cases	Coverage
NFR08 Compatibility	TC15	✓
NFR09 Portability	TC11–TC12	✓

6. Ensure that the test cases provide adequate requirement and functional coverage for the assigned system.

Metric	Result
Total Test Cases	**15**
Passed	**15**
Failed	**0**
Functional Tests	**12**
Non-Functional Tests	**3**
Boundary Tests	**2**
Equivalence Partition Tests	**3**
Decision Table Tests	**1**
State Transition Tests	**1**